

**PROJECT NAME : AWS LAMBDA S3 UPLOAD
NOTIFICATION**

NAME : ASHISH TALEKAR

DATE : 04/08/2026



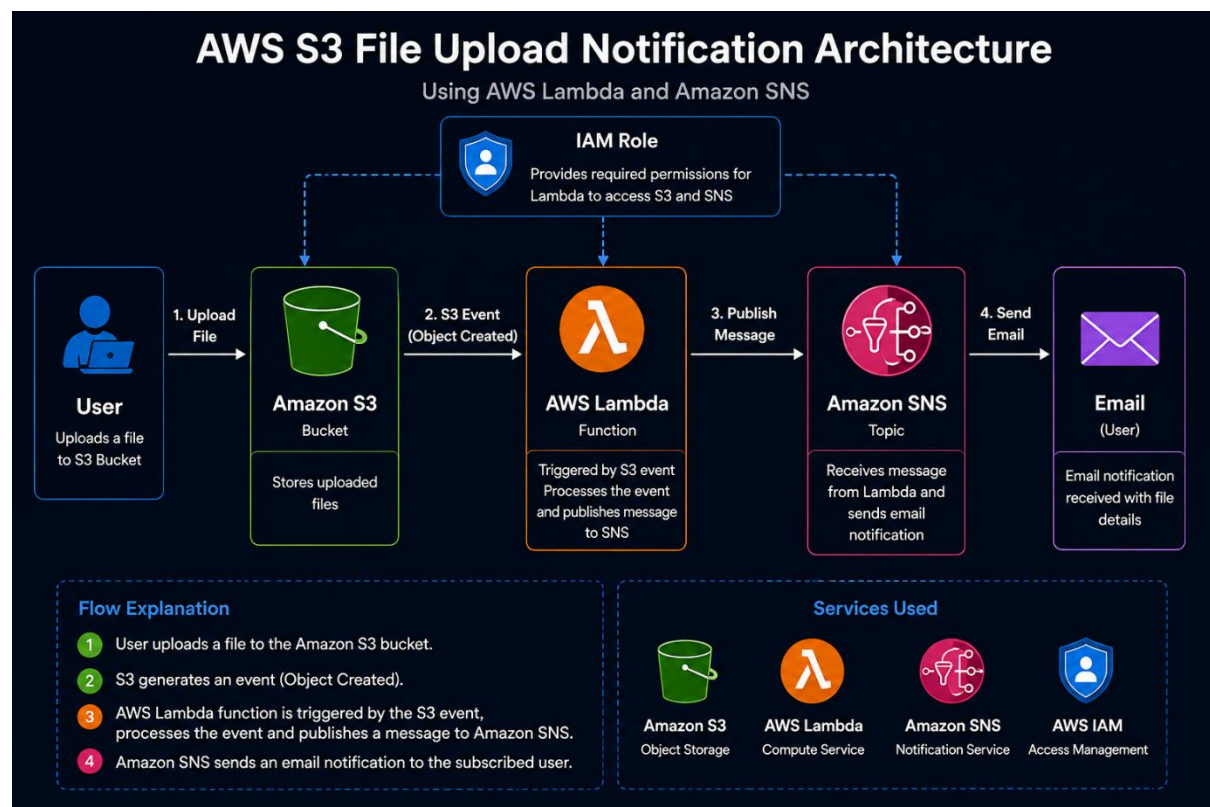
Project Overview

This project demonstrates an event-driven serverless architecture using AWS Lambda, Amazon S3, and Amazon SNS. Whenever a new file is uploaded to an Amazon S3 bucket, an S3 event automatically triggers an AWS Lambda function. The Lambda function processes the event details and publishes a notification to an Amazon SNS topic, which sends an email alert to the subscribed user. This project showcases the integration of AWS serverless services to automate file upload notifications without managing any servers.

Project Objective

The primary objective of this project is to gain hands-on experience with AWS serverless computing by integrating Amazon S3, AWS Lambda, and Amazon SNS. The project demonstrates how to automate workflows using event-driven architecture, configure AWS services securely, and deliver real-time email notifications whenever new files are uploaded to an S3 bucket.

Architecture Diagram :



AWS Services Used

1. Amazon S3 (Simple Storage Service)

Amazon S3 is an object storage service used to securely store files and generate event notifications. In this project, it stores uploaded files and automatically triggers the Lambda function whenever a new object is uploaded.

2. AWS Lambda

AWS Lambda is a serverless compute service that executes code in response to events without requiring server management. In this project, the Lambda function processes the S3 upload event and publishes a notification to Amazon SNS.

3. Amazon SNS (Simple Notification Service)

Amazon SNS is a fully managed messaging and notification service. It is used to send real-time email notifications containing the uploaded file details to the subscribed user.

4. IAM Role

IAM Role provides secure permissions that allow the Lambda function to access other AWS services. In this project, it grants the required permissions to read Amazon S3 events, publish messages to Amazon SNS, and write logs to Amazon CloudWatch.

5. Environment Variables

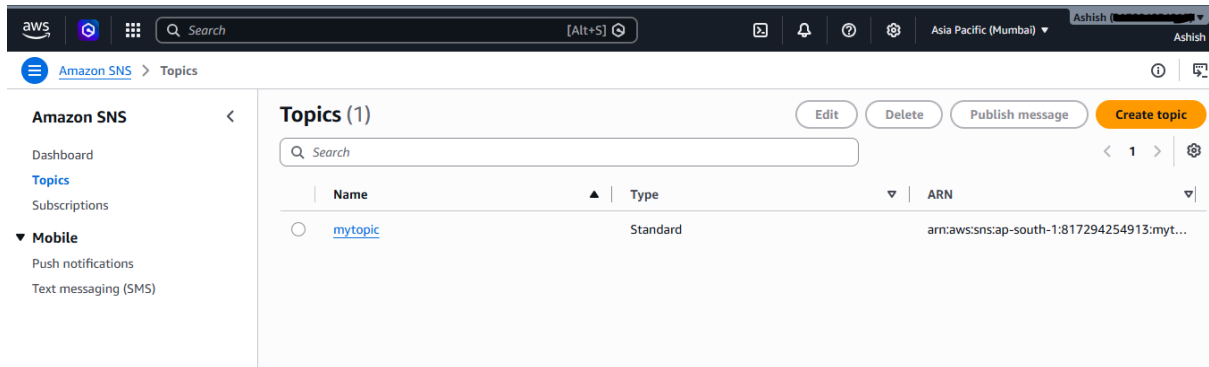
Environment Variables securely store configuration values without hardcoding them into the source code. In this project, the Amazon SNS Topic ARN is stored as an environment variable and accessed by the Lambda function during execution.

6. Amazon S3 Event Trigger

Amazon S3 Event Trigger automatically invokes the Lambda function whenever a specified event occurs. In this project, the Object Created (File Upload) event is configured to trigger the Lambda function after a new file is uploaded to the S3 bucket.

Configuration :

Step 1 - Create Amazon SNS Topic



Explanation :

An Amazon SNS topic was created to act as the messaging service for the notification system. This topic is responsible for receiving messages from the Lambda function and delivering them to subscribed users via email.

Step 2 - Configure Email Subscription

The screenshot shows the AWS Management Console interface for an Amazon SNS topic named 'mytopic'. The breadcrumb navigation at the top indicates the path: Amazon SNS > Topics > mytopic. The topic's details are displayed in a card, including its Name (mytopic), ARN (arn:aws:sns:ap-south-1:817294254913:mytopic), Display name (-), and Type (Standard). The Topic owner is listed as 817294254913. Below the details card, a horizontal menu allows switching between various settings: Subscriptions (selected), Access policy, Delivery policy (HTTP/S), Delivery status logging, Encryption, Tags, and Integrations. The 'Subscriptions (1)' section contains a search bar and a table with one subscription. The table has columns for ID, Endpoint, Status, and Protocol. The single subscription has an ID of 3efb5bac-18fa-438d-be31-..., an Endpoint of ashishtalekar0097@gmail.c..., a Status of Confirmed, and a Protocol of EMAIL. Action buttons for the subscription include Edit, Delete, Request confirmation, Confirm subscription, and a prominent Create subscription button.

mytopic [Edit](#) [Delete](#) [Publish message](#)

Details

Name
mytopic

ARN
arn:aws:sns:ap-south-1:817294254913:mytopic

Display name
-

Type
Standard

Topic owner
817294254913

[Subscriptions](#) | [Access policy](#) | [Delivery policy \(HTTP/S\)](#) | [Delivery status logging](#) | [Encryption](#) | [Tags](#) | [Integrations](#)

Subscriptions (1) [Edit](#) [Delete](#) [Request confirmation](#) [Confirm subscription](#) [Create subscription](#)

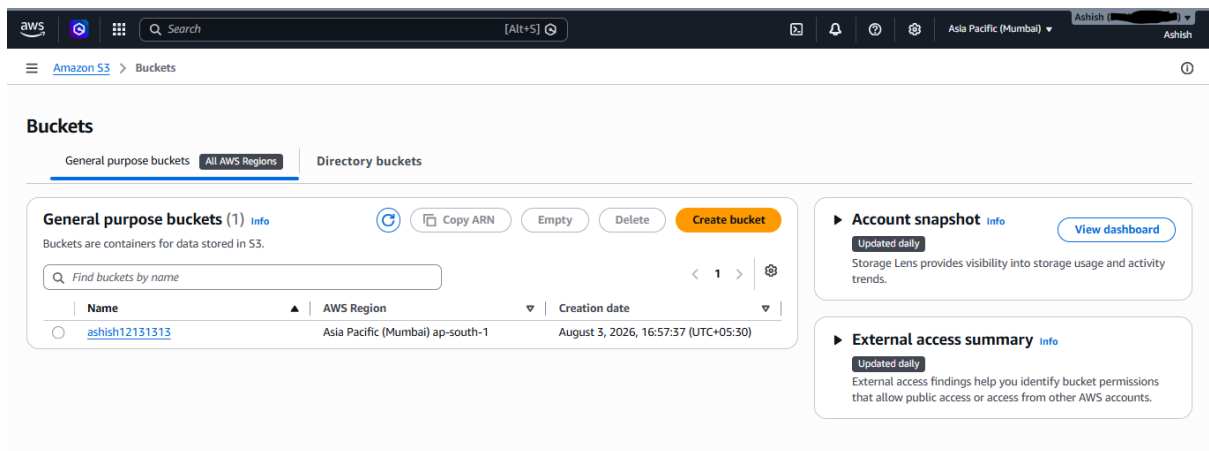
Q Search

ID	Endpoint	Status	Protocol
3efb5bac-18fa-438d-be31-...	ashishtalekar0097@gmail.c...	Confirmed	EMAIL

Explanation :

An email subscription was added to the SNS topic and confirmed by the recipient. This enables Amazon SNS to send notification emails whenever the Lambda function publishes a message.

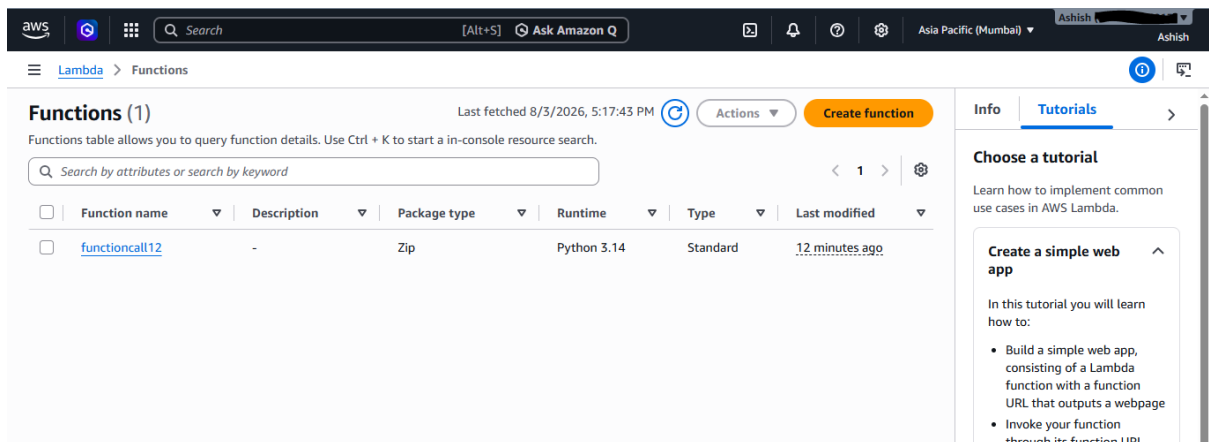
Step 3 - Create Amazon S3 Bucket



Explanation :

An Amazon S3 bucket was created to store uploaded files. This bucket serves as the event source and automatically triggers the Lambda function whenever a new object is uploaded.

Step 4 - Deploy AWS Lambda Function



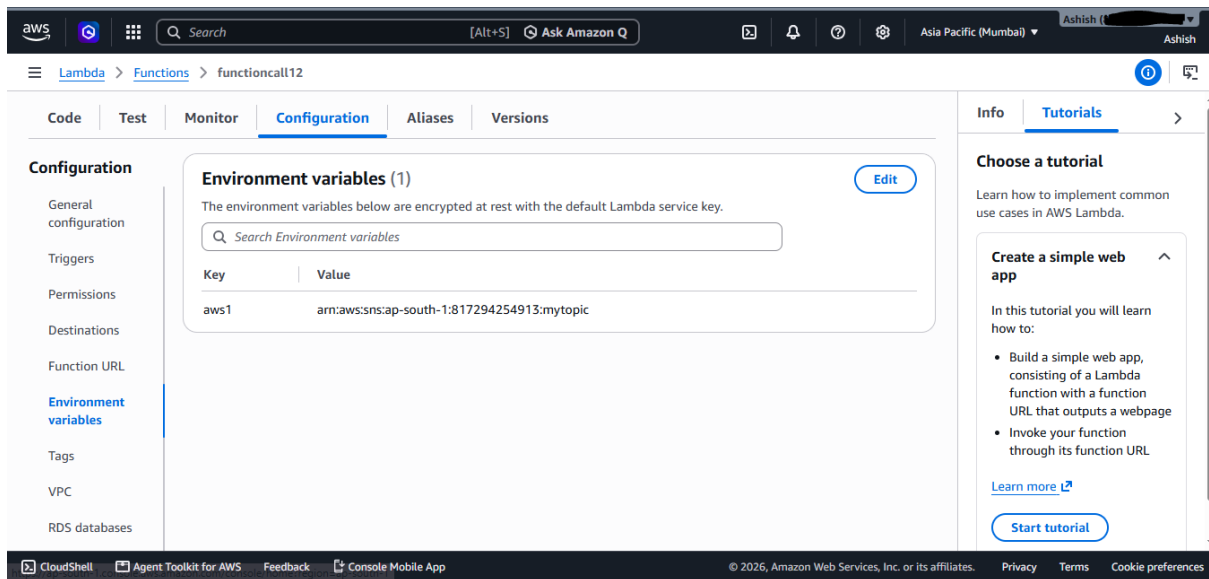
The screenshot shows the AWS Lambda console interface. At the top, there's a navigation bar with the AWS logo, a search bar, and a user profile. Below the navigation bar, the 'Functions' section is active, displaying a table with one function named 'functioncall12'. The table columns are: Function name, Description, Package type, Runtime, Type, and Last modified. The function 'functioncall12' has a description of '-', a package type of 'Zip', a runtime of 'Python 3.14', a type of 'Standard', and was last modified '12 minutes ago'. To the right of the table, there's a sidebar with 'Info' and 'Tutorials' tabs. The 'Tutorials' tab is selected, showing a 'Choose a tutorial' section with a link to 'Create a simple web app'.

Function name	Description	Package type	Runtime	Type	Last modified
functioncall12	-	Zip	Python 3.14	Standard	12 minutes ago

Explanation :

A new AWS Lambda function was created using the Python runtime. The function is responsible for processing S3 upload events and sending notification messages to Amazon SNS.

Step 5 - Configure Lambda Environment Variables



Explanation :

Environment Variables were configured to securely store the Amazon SNS Topic ARN. This allows the Lambda function to access configuration values without hardcoding them into the source code.

Step 6 - Assign IAM Execution Role

Configuration

General configuration
Triggers
Permissions
Destinations
Function URL
Environment variables
Tags
VPC
RDS databases
Monitoring and

Execution role

Role name
[functioncall12-role-xoeu0jlq](#)

Resource summary

To view the resources and actions that your function has permission to access, choose a service.

Amazon CloudWatch Logs
3 actions 2 resources

By action By resource

Resource	Actions
arn:aws:logs:ap-south-1:817294254913:*	Allow: logs:CreateLogGroup
arn:aws:logs:ap-south-1:817294254913:log-group:/aws/lambda/functioncall12:*	Allow: logs:CreateLogStream Allow: logs:PutLogEvents

Choose a tutorial

Learn how to implement common use cases in AWS Lambda.

Create a simple web app

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
- Invoke your function through its function URL

[Learn more](#)

[Start tutorial](#)

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

Roles
Policies
IAM users
IAM user groups
Identity providers
Account settings
Root access management
Temporary delegation requests

Access reports

Access Analyzer
Resource analysis

functioncall12-role-xoeu0jlq

Last activity
-

Maximum session duration
1 hour

Permissions Trust relationships Tags Last Accessed Revoke sessions

Permissions policies (2)

You can attach up to 10 managed policies.

Search

Filter by Type
All types

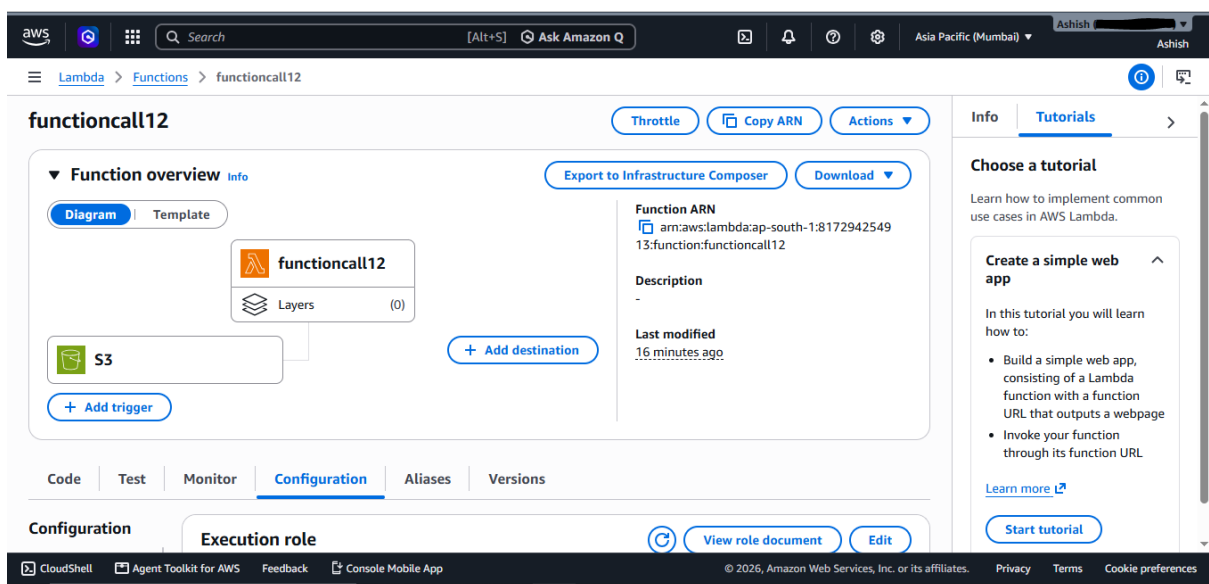
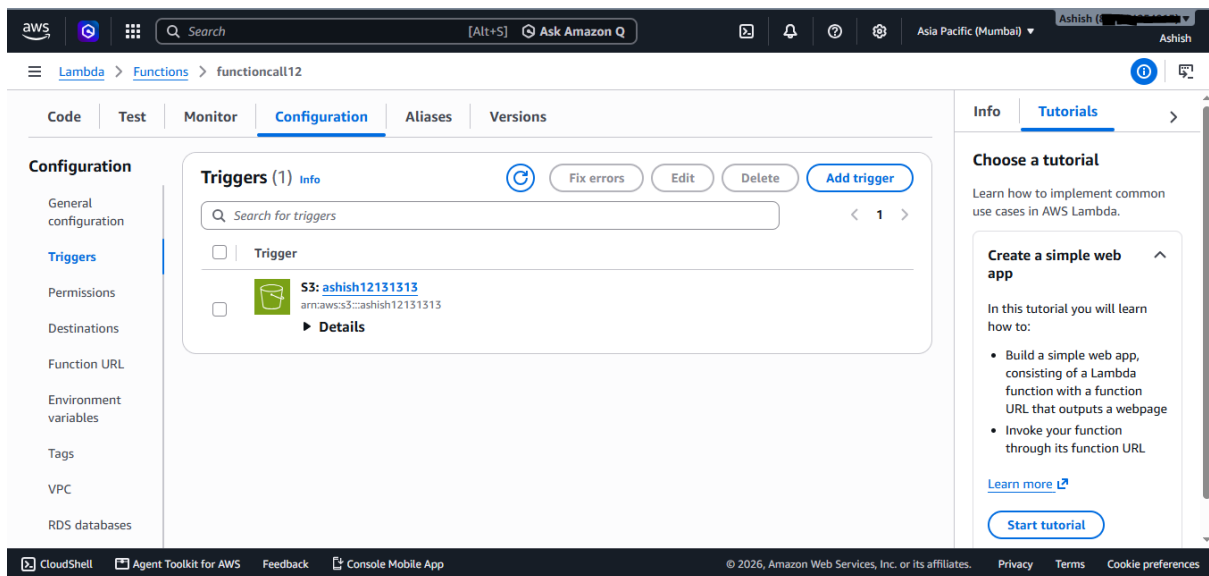
Policy name	Type	Attach...
AmazonSNSFullAccess	AWS managed	2
AWSLambdaBasicExecutionRole-622cd80e-56d5...	Customer managed	1

Permissions boundary (not set)

Explanation :

An IAM Execution Role with the required permissions was assigned to the Lambda function. The role allows the function to access Amazon S3, publish messages to Amazon SNS, and write logs to Amazon CloudWatch.

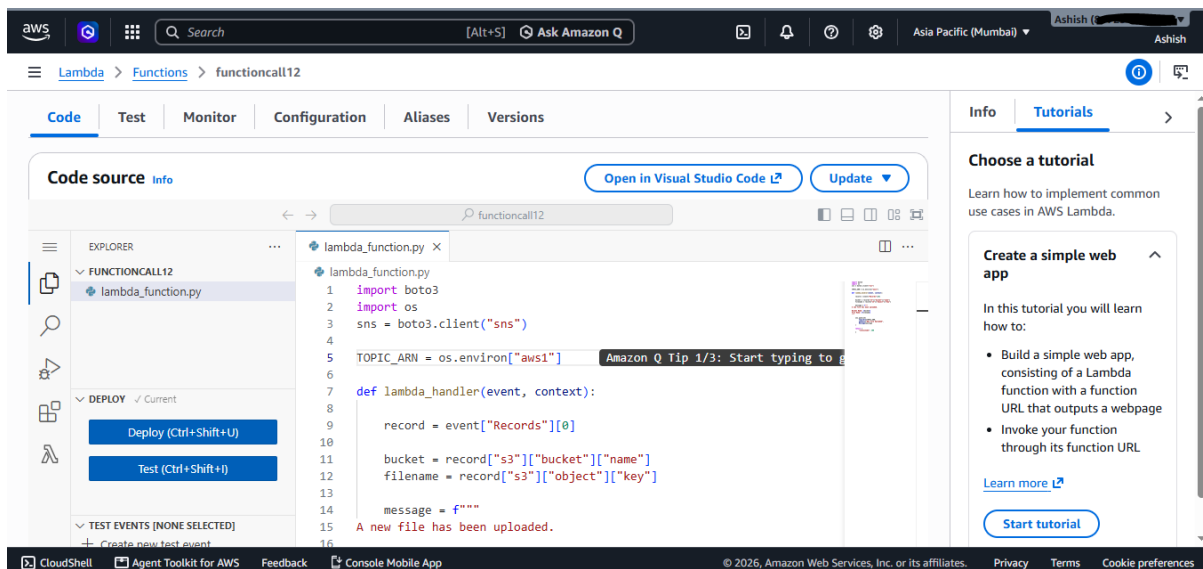
Step 7 - Configure Amazon S3 Event Trigger



Explanation :

An Amazon S3 Event Trigger was configured to invoke the Lambda function automatically whenever a new object is uploaded to the bucket. This enables event-driven automation without manual intervention.

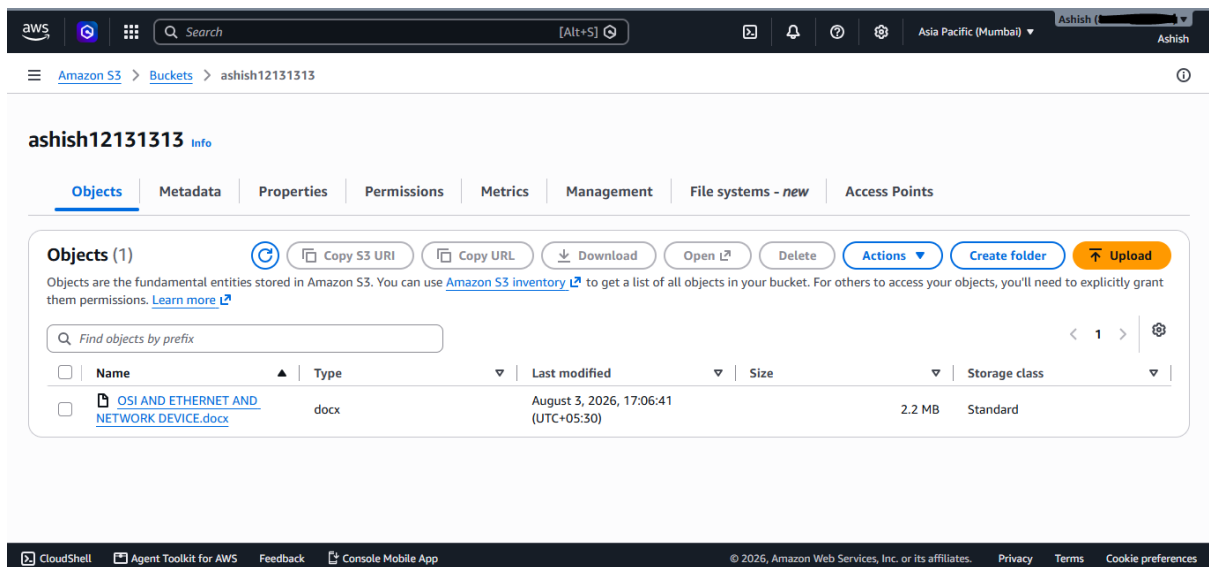
Step 8 - Implement and Deploy Lambda Function Code



Explanation :

The Python source code was implemented using the AWS Boto3 SDK to process S3 events and publish notification messages to Amazon SNS. The code was deployed successfully to the Lambda function.

Step 9 - Upload File to Amazon S3



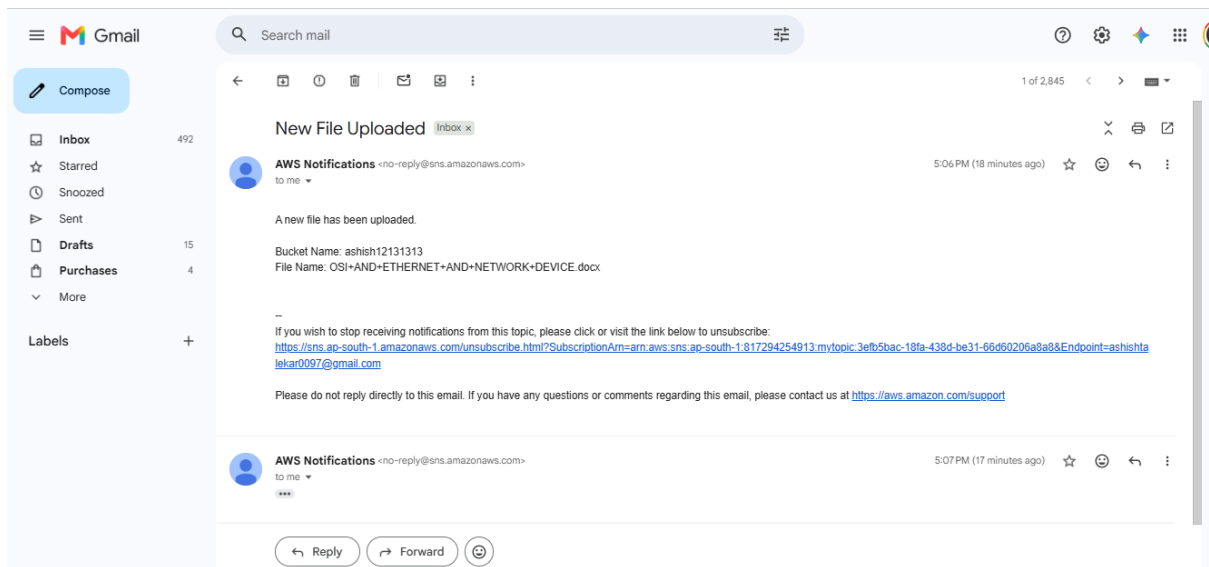
The screenshot displays the AWS Management Console interface for an Amazon S3 bucket named 'ashish12131313'. The console is in the 'Objects' tab, showing a list of objects. A single object, 'OSI AND ETHERNET AND NETWORK DEVICE.docx', is listed with a size of 2.2 MB and a storage class of 'Standard'. The object was last modified on August 3, 2026, at 17:06:41 (UTC+05:30). The console also shows various action buttons like 'Copy S3 URI', 'Copy URL', 'Download', 'Open', 'Delete', 'Actions', 'Create folder', and 'Upload'.

Name	Type	Last modified	Size	Storage class
OSI AND ETHERNET AND NETWORK DEVICE.docx	docx	August 3, 2026, 17:06:41 (UTC+05:30)	2.2 MB	Standard

Explanation :

A sample file was uploaded to the configured Amazon S3 bucket to test the event-driven workflow. The upload event successfully triggered the Lambda function for further processing.

Step 10 - Verify Email Notification



Explanation :

After the file upload, the Lambda function processed the event and published a message to Amazon SNS. The subscribed email address successfully received a notification containing the uploaded file details, confirming that the complete workflow was functioning correctly.

Key Features

- Created and configured an Amazon S3 bucket.
 - Created an Amazon SNS topic and confirmed an email subscription.
 - Developed an AWS Lambda function using Python (Boto3).
 - Configured Environment Variables for secure SNS Topic ARN management.
 - Assigned IAM Role with the required AWS permissions.
 - Configured an Amazon S3 Event Trigger to invoke the Lambda function automatically.
 - Uploaded files to Amazon S3 and triggered the Lambda function.
 - Sent real-time email notifications using Amazon SNS after every successful file upload.
-

Project Outcome

The project successfully implemented an automated file upload notification system using AWS serverless services. Every time a file was uploaded to the Amazon S3 bucket, the Lambda function was triggered automatically, processed the upload event, and published a notification to Amazon SNS. The subscribed user received an email containing the uploaded file name and bucket details, demonstrating a fully functional event-driven automation workflow.

Conclusion

This project successfully demonstrated the implementation of a serverless event-driven architecture using Amazon S3, AWS Lambda, and Amazon SNS. The integration of these AWS services enabled automatic processing of file upload events and real-time email notifications without the need to manage servers. Through this project, practical experience was gained in serverless computing, event-driven application design, IAM permission management, AWS service integration, and cloud automation concepts commonly used in modern cloud-based applications.